
Shaped By The Wind: Exploring Tabular Reinforcement Learning Methods

Dhanush Gowda Ramappa 5000343^{*1} Rahul Rao 4968964^{*1} Roshun Sai Chandrasekaran 4981731^{*1}

Abstract

This report evaluates fundamental tabular reinforcement learning methods in a Stochastic Windy Grid world. After establishing an optimal baseline using Dynamic Programming, we transition to model free settings to analyze the exploration exploitation dilemma via ϵ -greedy and Boltzmann (softmax) policies. We then contrast off-policy (Q learning) versus on-policy (SARSA) updates, and evaluate backup depths ranging from 1-step bootstrapping to full episode Monte Carlo. Our experiments illustrate the core bias variance trade-off, demonstrating how n -step methods bridge the gap between fast, biased 1-step updates and unbiased, high variance MC returns.

1. Introduction

Reinforcement Learning (RL) trains an agent to make decisions by maximizing rewards through trial and error. In this report, we explore these core principles using a Stochastic Windy Grid world. We start with Dynamic Programming (DP), which calculates perfect policies but unrealistically requires a complete rulebook of the environment. Shifting to more practical model free methods, we test ϵ -greedy and softmax exploration. We then compare off-policy (Q learning) with on-policy (SARSA) algorithms, and examine the bias variance trade off across 1-step, n -step, and Monte Carlo methods. Finally, we touch on real world limitations like the curse of dimensionality.

2. Background and Theory

2.1. Markov Decision Processes (MDPs)

RL problems are formalized as a Markov Decision Process, defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ (Bellman, 1966). Here, $\mathcal{S}$ represents the state space (all the possible situations the

agent can be in), and $\mathcal{A}$ represents the action space (all possible moves the agent can make). $\mathcal{P}(s_{t+1}|s_t, a_t)$ denotes the transition probability of reaching a new state s_{t+1} after taking action a_t in the current state s_t at timestep t . $\mathcal{R}(s, a, s')$ defines the immediate reward received after that specific transition, and $\gamma \in [0, 1]$ is the discount factor that determines the present importance of future rewards. The agent's objective is to find a policy π (a mapping from states to the probability of selecting an action) that maximizes the expected discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$.

2.2. Dynamic Programming (DP)

When the environment's transition dynamics ($\mathcal{P}$) and reward function ($\mathcal{R}$) are fully known, DP iteratively computes the optimal state action value function, $Q^*(s, a)$, which represents the maximum expected return achievable from state s by taking action a . Using Q value iteration, we sweep through the entire state space and apply the Bellman optimality equation (equation 1):

$$Q(s, a) \leftarrow \sum_{s'} \left[\mathcal{P}(s'|s, a) (\mathcal{R}(s, a, s') + \gamma \max_{a'} Q(s', a')) \right] \quad (1)$$

(Where s' and a' denote the possible subsequent state and subsequent action, respectively).

2.3. Action Selection and Exploration

In model free settings, agents must balance exploiting known high value actions with exploration of unknown ones. The ϵ -greedy policy ensures exploration by selecting a completely random action with probability ϵ and the greedy (highest Q value) action with probability $1 - \epsilon$. Alternatively, the Boltzmann (softmax) policy assigns selection probabilities based on current value estimates:

$$\pi(a|s) = \frac{e^{Q(s,a)/\tau}}{\sum_{b \in \mathcal{A}} e^{Q(s,b)/\tau}}$$

Here, $\tau > 0$ is the temperature parameter. A higher τ causes the policy to behave more randomly to encourage

^{*}Equal contribution. ¹LIACS, Leiden University, Netherlands. Correspondence to: Dhanush Gowda Ramappa <s5000343@vuw.leidenuniv.nl>.

exploration, while as $\tau \rightarrow 0$, the policy becomes strictly greedy.

2.4. Temporal Difference (TD) Learning

TD learning methods update Q values incrementally without requiring a perfect model, using an empirical target and a learning rate $\alpha \in (0, 1]$ which controls the step size (how heavily new information overrides old estimates). Q-learning (Watkins & Dayan, 1992) is an off-policy algorithm that updates its estimates using the maximum value of the next state, essentially learning the optimal policy regardless of the agent’s exploratory actions:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)] \quad (2)$$

Conversely, SARSA (Rummery & Niranjan, 1994) is an on-policy method that bootstraps using the value of the actual action a_{t+1} taken by the agent’s behavioral policy:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (3)$$

2.5. Backup Depth and Monte Carlo

The depth of the target backup dictates a crucial bias-variance tradeoff. 1-step TD methods bootstrap immediately, resulting in low variance but high bias. At the opposite extreme, Monte Carlo (MC) (Sutton & Barto, 1998) methods wait until episode is completed to use the full empirical return G_t without bootstrapping from other state estimates:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [G_t - Q(s_t, a_t)] \quad (4)$$

While MC is unbiased, it suffers from high variance in stochastic environments. The n -step Q learning algorithm bridges this gap by summing n real consecutive rewards before bootstrapping from the estimated value of state s_{t+n} , allowing for a tunable balance between bias and variance.

3. Experiments and Results

We have detailed all the experiments we have conducted in this section. (Graphs plotted in this report used a smoothing window of 27)

3.1. Dynamic Programming (DP)

We implemented Q value iteration to compute the optimal policy by systematically updating Q values using the Bell-

man equation and our known environment model. To understand how the algorithm learns, we visualized the Q-value estimates across the grid at the beginning, midway, and upon convergence.

At the start, the grid is mostly negative because every step costs -1; only states right next to the goal show positive values (as shown in Figure 1). Midway, this high value signal spreads across the grid. At convergence, the observed values make perfect sense: the state next to the goal is near 99.8, and as you work backwards, the values drop by about 1 for every step away from the target. It is observed that the start state’s value is $V(s = 3) = 83.7$. This means the maximum expected total return an agent can get from the start is 83.7. We also calculated the average reward per timestep, which is $83.7/18.3 \approx 4.57$.

In a subsequent test, moving the goal to the coordinates (6, 2) drastically dropped the start value to 64.9. Because each step costs -1, this result indicates that the shortest path now takes about 36 steps, completely doubling our original 18 step route.

3.2. Exploration

We compared ϵ -greedy ($\epsilon = 0.03, 0.1, 0.3$) and softmax policies (temperature $\tau = 0.01, 0.1, 1.0$). We ran each setting for 50,001 timesteps, evaluating every 1,000 steps, and averaged our results over 20 repetitions to produce learning curves, as in Figure 2.

In our results, both methods learned effectively, but we slightly prefer softmax ($\tau = 0.01$) because it converged slightly faster and smoother. However, we observed that our RL agents never perfectly reached the optimal DP performance.

In subsequent tests, adding a second, smaller goal (+5 reward) severely ruined overall performance, with returns flatlining near zero. However, changing the default step penalty from -1 to 0 did not help either; the agent wandered aimlessly and failed to reach the optimal return within the given timesteps.

3.3. Back-up, On-policy versus Off-policy

In our experiments (Figure 3), Q-learning consistently outperformed SARSA, proving to be better at higher learning rates. At $\alpha = 0.3$, Q-learning stabilized around a return of 60, whereas SARSA crashed to roughly 15. We prefer Q-learning for this specific gridworld task because it successfully finds the optimal path despite the stochastic wind. However, we would prefer SARSA in high risk environments (like cliff walking) because its on-policy nature inherently learns safer more conservative routes.

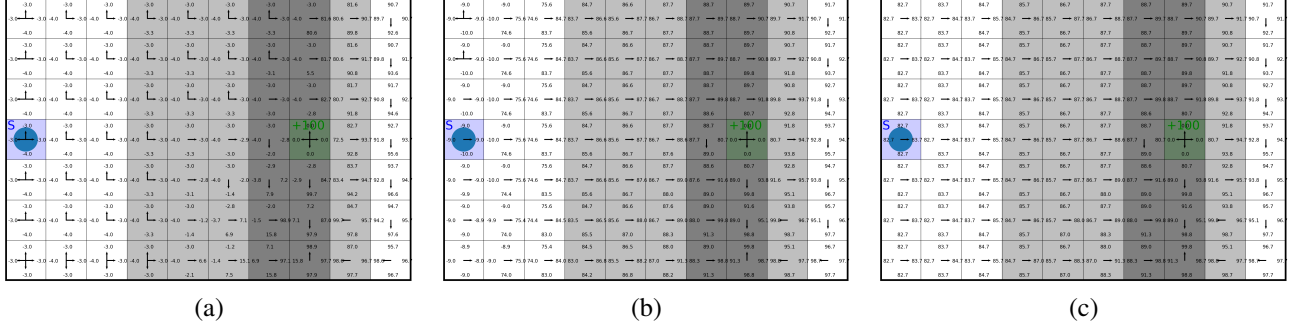


Figure 1. Progression of Q-value iteration showing the grid at the beginning (a), midway (b), and at convergence (c).

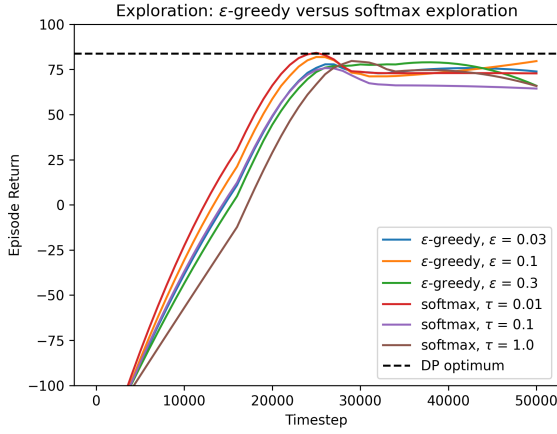


Figure 2. Comparison of ϵ -greedy and softmax exploration strategies across various parameter settings

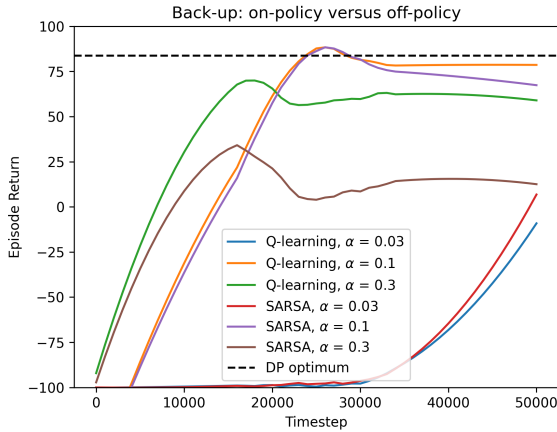


Figure 3. Comparison of SARSA and Q-learning across varying learning rates ($\alpha \in \{0.3, 0.1, 0.03\}$), (averaged over 20 reps)

3.4. Back-up Depth of Target

To understand the effect of backup depth on learning efficiency, we experimented with n -step Q-learning using depths of $n = 1, 3$, and 10, alongside the full Monte Carlo (MC) method. We observed the deeper backup methods (3-step and 10-step) learn faster initially (Figure 4). This happens because they propagate delayed reward information backward to earlier states much quicker within a single episode. However, the 3-step method achieves the best final performance, as it is the only one that stably converges on the optimal policy.

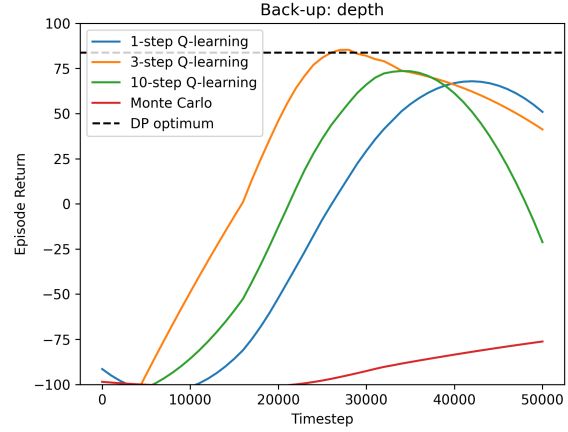


Figure 4. Comparison of different backup depths ($n = 1, 3, 10$) and Monte Carlo, (averaged over 20 reps)

4. Discussion

Analyzing results from Section 3.1, the drastic drop in the start value when moving the goal to (6, 2) occurs because the new goal sits in the strongest vertical wind column, which pushes the agent up by 2 squares every step hence, the agent is forced to take a much longer inefficient route.

Regarding the algorithm’s architecture, our DP code effectively stops infinite rewards at the goal by making it an absorbing state: the goal loops back to itself with a 100% probability and a 0 reward. As an alternative, another way to solve this without changing the environment model is to add an if done: check in the DP source file to strictly force the next state’s target value to 0. Furthermore, DP’s main strength is that it uses a perfect environment model to mathematically guarantee finding the optimal policy. Its critical weakness is that it completely relies on having this perfect model, which is rarely possible in real-world.

Analyzing the learning curves from Section 3.2, the gap between our RL agents and the optimal DP performance happens because fixed ϵ or τ values force the agent to continuously take random, sub-optimal steps even after finding the best path. A better way to explore would be an “annealing” strategy (Sutton & Barto, 2018), gradually decaying the exploration parameter to zero over time so the agent purely exploits the optimal path at the end. Considering addition of a second, smaller goal, drastically affected the exploration-exploitation trade off. The agent quickly found and exploited the nearby +5 goal, getting trapped in a local optimum. To overcome this, much higher exploration rates became necessary to force the agent to keep searching.

We also observed a failure when changing the default step penalty to 0. previously, all Q-values were initialized at 0, a standard -1 step penalty made visited states negative This encouraged the agent to explore unknown areas. When we changed the reward to 0 in our trials, the agent lost its urgency to find the shortest path and wandered aimlessly without the mathematical incentive to explore unvisited states.

In Section 3.3, Q-learning (off-policy) assumes optimal future actions, yielding stable convergence even at $\alpha = 0.3$ by ignoring its own exploratory mistakes. Conversely, SARSA (on-policy) evaluates the actual ϵ -greedy actions taken. While this finds safer routes, it causes severe instability at high learning rates by heavily penalizing itself for random exploration in the windy environment. Our n -step Q-learning is an off-policy method. Even though it collects real on-policy rewards for n steps, its final bootstrapping update uses the maximum Q-value of the future state ($\text{np.max}(\text{self.Q_sa}[\text{states}[\text{t} + \text{m}]])$).

Section 3.4 demonstrates the bias-variance trade-off: 1-step Q-learning (high bias, low variance) learns smoothly but slowly, while Monte Carlo (zero bias, high variance) is derailed by accumulated wind noise. The 3-step method hits the optimal sweet spot between these extremes. A performance gap exists between standard Q-learning and our n -step method at $n = 1$. This occurs because standard Q-learning updates online (immediately after each step), whereas our n -step implementation updates offline.

Experimental graphs of different discount factor suggest setting the step reward to 0 prevents the agent from learning the shortest path, failing to reach the optimal DP baseline. At $\gamma = 1.0$, the agent has no incentive to stop wandering. While $\gamma = 0.99$ theoretically penalizes longer routes through discounting, removing the immediate -1 step penalty creates a severe sparse reward problem. The discounted goal signal alone is too weak to guide the agent, proving that an explicit negative step penalty is essential for efficient learning.

5. Conclusion

In our experiments, tabular RL worked perfectly because our gridworld was simple, allowing our agent to easily memorize a Q-value for every single $[x, y]$ coordinate. However, if we were to make this environment more realistic, for instance, by adding continuous wind speeds or moving obstacles, we would quickly run into the “curse of dimensionality.” The number of possible states would explode exponentially, making our lookup tables completely impossible to store in memory or practically explore. We learned that deep learning helps overcome this issue by replacing these massive tables with neural networks. Instead of memorizing every specific grid square, the network learns the environment’s general patterns, allowing the agent to estimate the best actions even in completely unseen situations.

In conclusion, our gridworld experiments taught us the essential tradeoffs in reinforcement learning. While Dynamic Programming mathematically guarantees a perfect path, it unrealistically requires a flawless, preknown model of the environment. In our model-free tests, off-policy Q learning proved much more stable against the random wind than on-policy SARSA, which penalized itself too heavily for its own exploratory mistakes. Furthermore, our target depth analysis showed that 3-step Q learning hits the perfect “sweet spot,” learning much faster than 1-step methods while avoiding the massive variance of full Monte Carlo rollouts. Ultimately, this project showed us that successfully training an RL agent requires carefully balancing our choice of algorithm, exploration strategy, and backup depth to overcome environmental noise.

References

- Bellman, R. Dynamic programming. *Science*, 153(3731): 34–37, 1966.
- Rummery, G. A. and Niranjan, M. On-line Q-learning using connectionist systems. Technical report, Department of Engineering, University of Cambridge, Cambridge, UK, 1994.

Sutton, R. S. and Barto, A. G. *Reinforcement learning: An*

introduction, volume 1. MIT Press, Cambridge, MA, 1998.

Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

Watkins, C. J. C. H. and Dayan, P. Q-learning. *Machine Learning*, 8(3):279–292, 1992.